



VirtuStack

Juan Carrasco Milla

Alejandro Burruezo Cañadas

Victor Manuel Andreu Felipe

11 de junio de 2025

11/06/2025

Índice

Resumen	2
Contextualización del Proyecto	2
Justificación de la Elección del Tema	2
Objetivos del Proyecto	2
Marco Teórico	3
Metodología	3
Infraestructura	4
Proxy nginx	5
Certificado SSL	6
1. Requisitos previos	6
2. Iniciar el proceso con Certbot	6
3. Gestionar el registro TXT en DuckDNS	6
3.1. Borrar el registro TXT anterior (opcional pero recomendable):	6
3.2. Crear el nuevo registro TXT con el valor que te da Certbot:	6
4. Verificar la propagación del registro TXT	6
5. Completar el proceso en Certbot	6
Portainer	7
Wireguard	12
DDNS	13
Cloud init	13
Gitlab	13
Plataforma web	15
Resultados	18
AWX	20
1. Requisitos previos	20
2. Instalación de paquetes	20
3. Instalamos Kubernetes con k3s	20
4. Instalamos Kustomize	20
5. Instalamos AWX Operator	20
6. Instalamos la instancia de AWX	21
7. Resultado final:	23
Github	28
Playbooks	28
Playbook de Oracle	28
Playbook de WordPress	28
Playbook de HAProxy	28
Playbook de Router (Kea y BIND)	29
Playbook de Migración de Máquinas Virtuales	29
Resultados	29

Webgrafía: 30

10. Anexos 30

Resumen

VirtuStack desarrolla una solución integral para la gestión y sincronización automatizada de máquinas virtuales y usuarios en un entorno de infraestructura IT, utilizando un dashboard web propio y la plataforma de automatización AWX. La aplicación, construida con tecnologías modernas como Next.js, React, Node.js y MySQL, permite a los administradores crear, monitorizar y sincronizar recursos de forma segura y eficiente, integrando autenticación, ejecución remota de comandos con asistente de IA y gestión centralizada de credenciales y usuarios.

Contextualización del Proyecto

En entornos IT modernos, la gestión eficiente de recursos virtualizados y usuarios es fundamental para garantizar la seguridad, la escalabilidad y la operatividad de los servicios. Plataformas como AWX (la versión open-source de Ansible Tower) facilitan la automatización de tareas, pero muchas organizaciones requieren interfaces personalizadas que se adapten a sus flujos de trabajo y sistemas existentes. Este proyecto surge para cubrir esa necesidad, proporcionando una capa de integración entre AWX y una aplicación web propia que centraliza la administración de máquinas virtuales y usuarios.

Justificación de la Elección del Tema

La automatización y la administración centralizada de infraestructuras virtuales son tendencias clave en la evolución de los sistemas IT. Sin embargo, la integración entre herramientas de automatización como AWX y aplicaciones personalizadas no siempre es trivial, especialmente cuando se requiere una gestión segura de usuarios, contraseñas y recursos virtuales. Elegir este tema permite abordar un reto real en la industria, aplicando tecnologías actuales y buenas prácticas de desarrollo seguro y escalable.

Objetivos del Proyecto

- Objetivo General

Desarrollar una plataforma web que integre la gestión de máquinas virtuales y su administración mediante IA y usuarios con AWX, permitiendo la automatización y sincronización de recursos de forma segura y eficiente.

- Objetivos Específicos

- Implementar un dashboard web intuitivo para la creación y gestión de máquinas virtuales y usuarios.
- Integrar la aplicación con AWX mediante su API REST, automatizando la creación y asociación de usuarios y organizaciones.
- Garantizar la seguridad en el almacenamiento y transmisión de credenciales, empleando hash bcrypt para contraseñas.
- Ejecutar comandos remotos en las VMs mediante SSH y reflejar los resultados en tiempo real con la asistencia de Gemini
- Sincronizar el estado de las VMs y usuarios entre la base de datos local y AWX.
- Proporcionar un sistema robusto de manejo de errores y validación tanto en frontend como en backend.
- Aprovechar el potencial de Ansible como herramienta de automatización mediante la creación de playbooks de infraestructura y despliegue.

Marco Teórico

Virtualización: Tecnología que permite ejecutar múltiples sistemas operativos en una sola máquina física, optimizando recursos y facilitando la administración.

Proxmox: Plataforma de virtualización de código abierto que permite la gestión centralizada de máquinas virtuales y contenedores.

Automatización IT: Uso de herramientas como Ansible/AWX para la orquestación y automatización de tareas de infraestructura, mejorando la eficiencia y reduciendo errores humanos.

Ansible: Herramienta de automatización de TI que permite configurar y desplegar infraestructuras mediante playbooks escritos en YAML, facilitando la estandarización y la escalabilidad de los despliegues.

AWX: Plataforma open-source que proporciona una interfaz web y API REST para gestionar y automatizar tareas con Ansible.

Node.js y Next.js: Entorno y framework para el desarrollo de aplicaciones web, permitiendo la integración de backend y frontend en un mismo proyecto.

MySQL: Sistema de gestión de bases de datos relacional, empleado para almacenar información de usuarios, máquinas virtuales y credenciales.

SSH: Protocolo seguro para la administración remota de sistemas, utilizado aquí para ejecutar comandos en las VMs.

WireGuard: Protocolo de VPN que ofrece una conexión segura y eficiente entre redes, utilizado para habilitar el acceso remoto y la comunicación cifrada entre los distintos componentes del proyecto.

Metodología

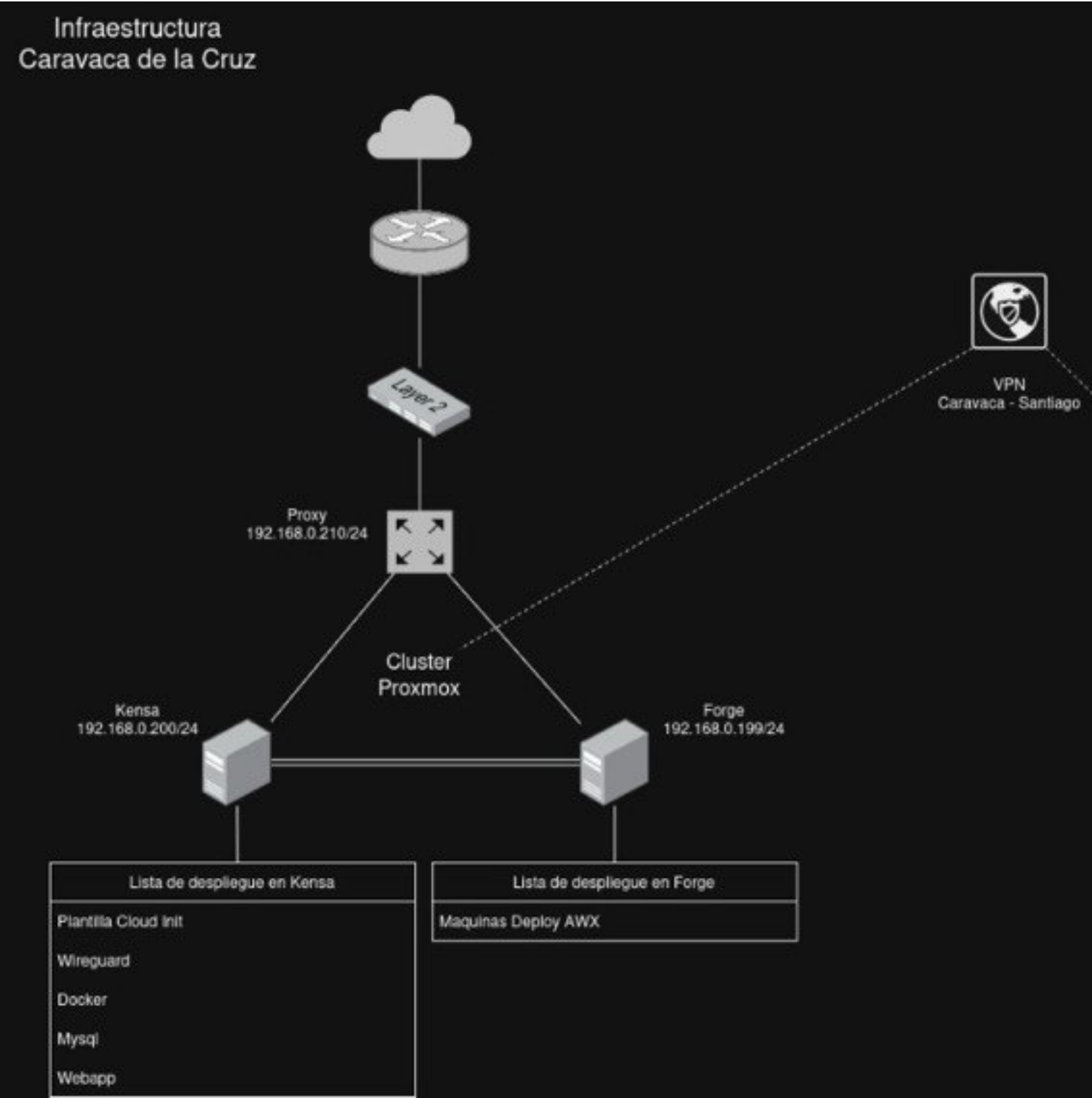
Análisis de requisitos: Identificación de necesidades de gestión y sincronización entre la aplicación y AWX.

Diseño de arquitectura: Definición de la estructura de la base de datos, endpoints de la API y flujos de integración con AWX.

Desarrollo incremental: Implementación iterativa de funcionalidades, comenzando por la gestión básica de VMs y usuarios, y añadiendo integración con AWX y SSH.

Pruebas y validación: Verificación funcional de los endpoints, pruebas de seguridad (hash de contraseñas, manejo de errores) y validación de la integración con AWX.

Documentación y entrega: Elaboración de documentación técnica para la correcta instalación, uso y mantenimiento de la infraestructura.



Proxy nginx

Configuración de proxy

```
1 server {
2     listen 80;
3     server_name cromopolis.duckdns.org;
4     return 301 https://$host$request_uri;
5 }
6 server {
7     listen 443 ssl;
8     server_name cromopolis.duckdns.org;
9
10    ssl_certificate /etc/letsencrypt/live/cromopolis.duckdns.org/fullchain.pem;
11    ssl_certificate_key /etc/letsencrypt/live/cromopolis.duckdns.org/privkey.pem;
12
13    ssl_protocols TLSv1.2 TLSv1.3;
14    ssl_ciphers HIGH:!aNULL:!MD5;
15
16    # Proxy para WebSocket SSH interactivo
17    location /ws/ {
18        proxy_pass http://192.168.0.194:8080/;
19        proxy_http_version 1.1;
20        proxy_set_header Upgrade $http_upgrade;
21        proxy_set_header Connection "Upgrade";
22        proxy_set_header Host $host;
23        proxy_read_timeout 3600;
24    }
25    # Proxy para tu app Next.js
26    location / {
27        proxy_pass http://192.168.0.194:3000;
28        proxy_set_header Host $host;
29        proxy_set_header X-Real-IP $remote_addr;
30        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
31        proxy_set_header X-Forwarded-Proto $scheme;
32    }
33 }
```

Certificado SSL

1. Requisitos previos

- Dominio DuckDNS activo (ejemplo: cromopolis.duckdns.org)
- Token DuckDNS
- Servidor Linux con acceso a internet
- Certbot instalado

2. Iniciar el proceso con Certbot

Ejecutar el siguiente comando para iniciar la solicitud del certificado:

```
1 sudo certbot certonly --manual --preferred-challenges dns -d cromopolis.duckdns.org
```

Certbot mostrará un mensaje parecido a este:

```
1 Please deploy a DNS TXT record under the name:
2
3 _acme-challenge.cromopolis.duckdns.org.
4
5 with the following value:
6
7 VALOR_UNICO_QUE_TE_DA_CERTBOT
```

3. Gestionar el registro TXT en DuckDNS

3.1. Borrar el registro TXT anterior (opcional pero recomendable):

curl "https://www.duckdns.org/update?domains=cromopolis&token=TU_TOKEN&clear=true"

3.2. Crear el nuevo registro TXT con el valor que te da Certbot:

curl "https://www.duckdns.org/update?domains=cromopolis&token=TU_TOKEN&txt=VALOR_UNICO_QUE_TE_DA_CERTBOT"

- Sustituye TU_TOKEN por tu token real de DuckDNS.
- Sustituye VALOR_UNICO_QUE_TE_DA_CERTBOT por el valor exacto que te muestra Certbot.

4. Verificar la propagación del registro TXT

Antes de continuar, verifica que el registro TXT esté visible en servidores DNS públicos:

```
dig _acme-challenge.cromopolis.duckdns.org TXT @8.8.8.8 dig _acme-challenge.cromopolis.duckdns.org TXT @1.1.1.1
dig _acme-challenge.cromopolis.duckdns.org TXT @9.9.9.9
```

5. Completar el proceso en Certbot

Cuando el registro TXT esté correctamente propagado y visible en los DNS públicos, vuelve a la terminal donde Certbot espera y pulsa *Enter*.

Si todo está correcto, Certbot mostrará un mensaje de éxito y los archivos generados:

- Certificado: `/etc/letsencrypt/live/cromopolis.duckdns.org/fullchain.pem`
- Clave privada: `/etc/letsencrypt/live/cromopolis.duckdns.org/privkey.pem`

Portainer

Plataforma en la que se encuentran los contenedores docker. Contenedor que comunica con DuckDns para el cambio de IP pública

Upgrade to Business Edition



portainer.io
COMMUNITY EDITION



Home



local



Dashboard



Templates



Stacks



Containers



Images



Networks



Volumes



Events



Host



Administration



User-related



Environment-related



Registries



Logs



Notifications



Settings



Environment summary

Dashboard



Environment info

Environment

URL

GPU

Tags



2

Stacks



7

Images



4

Networks



portainer.io
COMMUNITY EDITION



Home



local



Dashboard



Templates



Stacks



Containers



Images



Networks



Volumes



Events



Host



Administration



User-related



Environment-related



Registries



Logs



Notifications



Settings



Container details



Actions



Start



Stop



Kill



Restart



Container status

ID

Name

Status

Created

Start time

Container webhook ?

Logs

Inspect

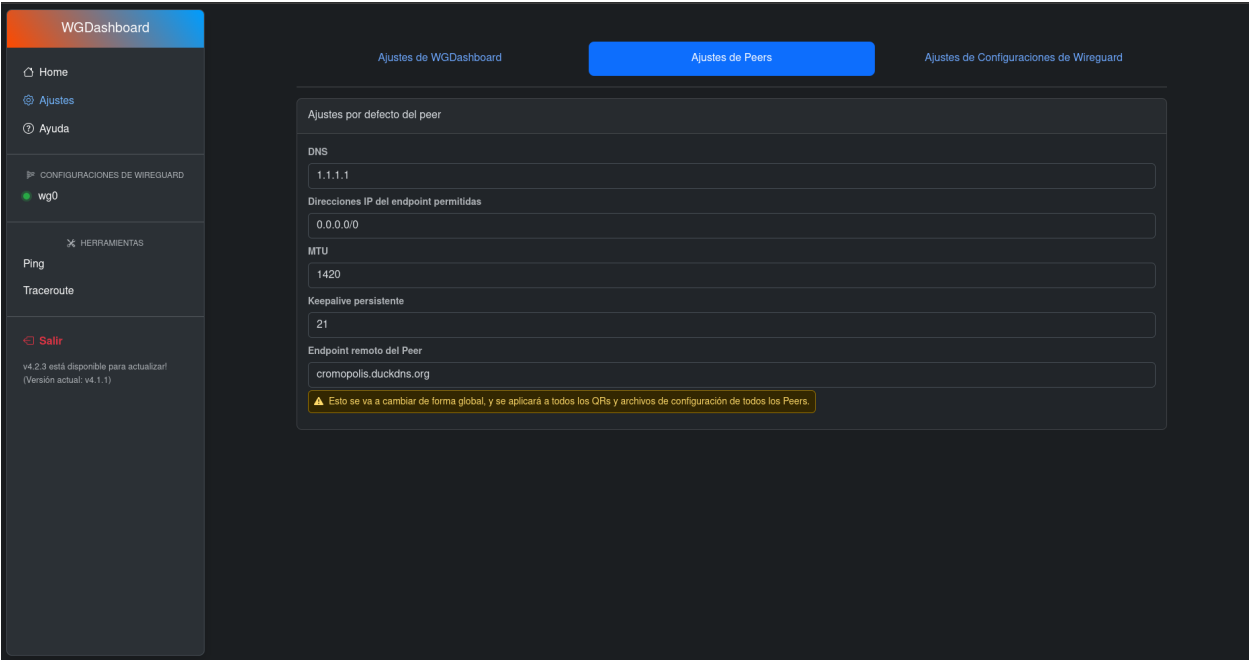
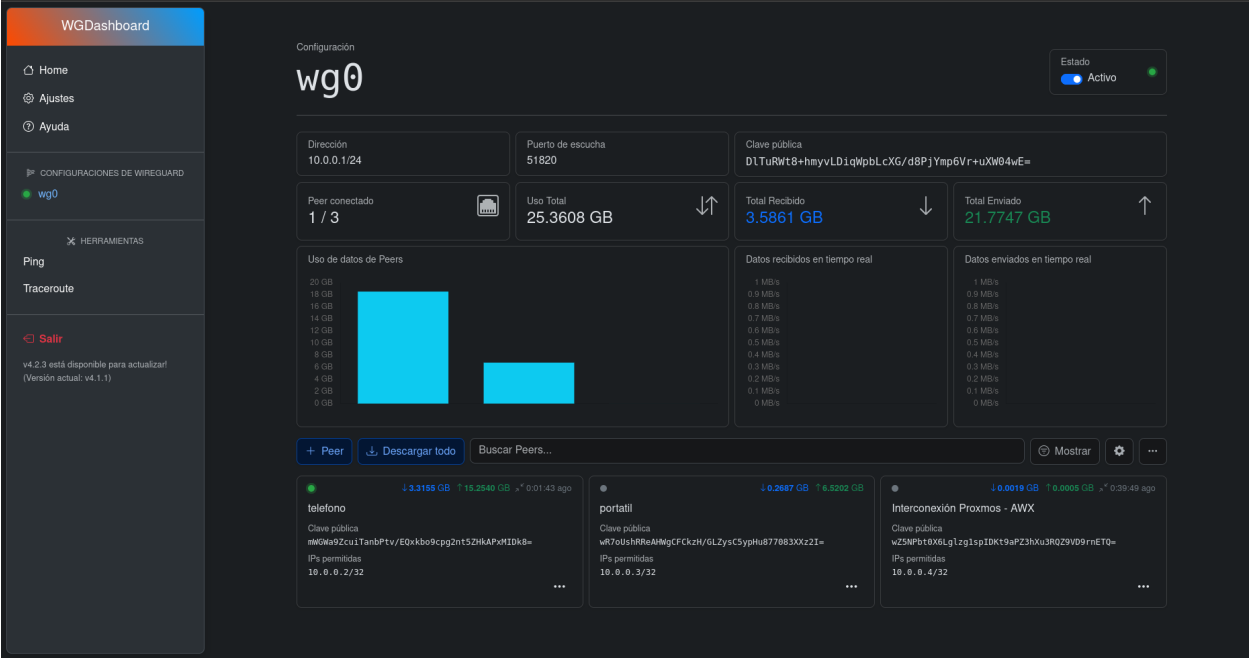
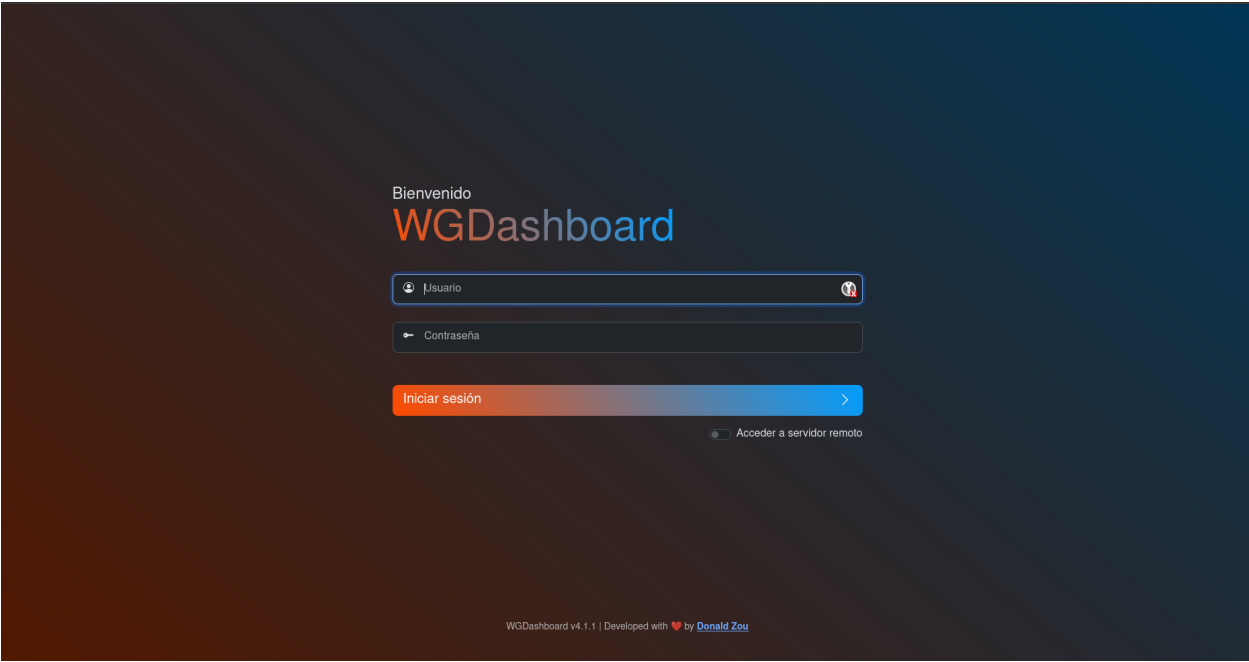
Stats



Access control

Ownership

```
1 services:
2   duckdns:
3     image: lscr.io/linuxserver/duckdns:latest
4     container_name: duckdns
5     network_mode: host #optional
6     environment:
7       - PUID=1000 #optional
8       - PGID=1000 #optional
9       - TZ=Etc/UTC #optional
10      - SUBDOMAINS=cromopolis
11      - TOKEN=x
12      - LOG_FILE=false #optional
13     volumes:
14       - /root/docker/duckdns/config:/config #optional
15     restart: unless-stopped
```

DDNS

Duck DNS se utiliza como servicio de DNS dinámico (DDNS) para asociar un subdominio a la IP pública de nuestra red, lo que resulta esencial cuando no disponemos de una IP fija. En este proyecto, se emplea un contenedor de Docker que actualiza automáticamente la IP pública registrada en Duck DNS cada pocos minutos, asegurando que el subdominio apunte siempre a la dirección correcta incluso si la IP cambia. De este modo, es posible acceder de forma remota y sencilla a los servicios desplegados en la red local, sin preocuparse por los cambios de dirección IP asignados por el proveedor de Internet

Cloud init

Cloud-init es una herramienta ampliamente utilizada para automatizar la configuración inicial de máquinas virtuales Linux en entornos de nube o virtualización. Permite personalizar una instancia durante su primer arranque (o en arranques posteriores) aplicando configuraciones como la creación de usuarios, la instalación de paquetes, la configuración de redes, la asignación de claves SSH, o la ejecución de scripts personalizados

Gitlab

GitLab es una plataforma de gestión de código fuente y control de versiones basada en Git, ampliamente utilizada para almacenar, organizar y colaborar en proyectos de software. En este proyecto, GitLab se emplea como repositorio centralizado donde se almacena todo el código fuente del dashboard, el backend y los scripts de integración con AWX, lo que facilita el trabajo colaborativo y el seguimiento de los cambios realizados a lo largo del desarrollo

El uso de GitLab permite aprovechar el control de versiones distribuido, de modo que cada desarrollador puede trabajar con una copia local del proyecto, crear ramas para nuevas funcionalidades o correcciones, y fusionar los cambios de forma segura y transparente en la rama principal

Plataforma web

```
— eslint.config.mjs
— hash-password.ts
— next.config.ts
— next-env.d.ts
— package.json
— package-lock.json
— postcss.config.mjs
— public
  — file.svg
  — globe.svg
  — next.svg
  — vercel.svg
  — window.svg
— README.md
— src
  — app
    — api
      — auth
        — [...nextauth]
          — route.ts
      — create-vm
        — route.ts
      — login
        — route.ts
      — logout
        — route.ts
      — me
        — route.ts
      — register
        — route.ts
      — sync-awx
        — route.ts
      — vms
        — route.ts
        — [vmid]
          — exec
            — route.ts
          — metrics
            — route.ts
          — mysql-action
            — route.ts
          — reboot
            — route.ts
          — route.ts
          — start
            — route.ts
          — stop
            — route.ts
          — suggest
            — route.ts
    — create-vm
      — page.tsx
    — dashboard
      — page.tsx
    — favicon.ico
    — globals.css
    — layout.tsx
    — login
      — page.tsx
    — manage-vms
      — page.tsx
      — [vmid]
        — admin
          — Console.tsx
          — page.tsx
        — page.tsx
        — suggest
          — page.tsx
    — page.tsx
    — register
      — page.tsx
  — components
    — AutomationsSection.tsx
    — CTA.tsx
    — Features.tsx
    — Footer.tsx
    — Hero.tsx
    — Navbar.tsx
    — TechStack.tsx
    — ui
      — badge.tsx
      — button.tsx
      — card.tsx
      — tabs.tsx
      — textarea.tsx
    — VmCpuChart.tsx
    — VMList.tsx
  — lib
    — db.ts
    — proxmox.ts
    — utils.ts
  — middleware.ts
  — ssh-ws-server.js
  — types
    — nextauth.d.ts
  — tsconfig.json
```

Este proyecto es una plataforma web para la gestión y automatización de máquinas virtuales y usuarios, integrando servicios como AWX y Proxmox. Está desarrollado con Next.js (React) y Node.js, y utiliza una arquitectura modular con separación clara entre frontend, backend (API), componentes y utilidades.

Frontend: Las páginas de usuario, como el dashboard, login, registro y la creación y gestión de VMs, se encuentran en `src/app/`. Estas páginas interactúan con la API mediante formularios y peticiones HTTP.

Backend/API: La lógica de negocio y las integraciones externas residen en los endpoints de la API, ubicados en `src/app/api/`. Aquí se gestionan operaciones como autenticación, registro de usuarios, creación de VMs, sincronización con AWX, y acciones sobre las máquinas virtuales (iniciar, detener, reiniciar, obtener métricas, ejecutar comandos, etc.).

Componentes reutilizables: En `src/components/` se encuentran los elementos de interfaz reutilizables y secciones del dashboard, como tablas de VMs, gráficos de uso de CPU, formularios y elementos UI personalizados.

Integración y utilidades: El directorio `src/lib/` contiene la lógica de conexión con la base de datos (`db.ts`), integración con Proxmox (`proxmox.ts`), y funciones auxiliares (`utils.ts`).

Gestión de autenticación: Se utiliza NextAuth para la autenticación de usuarios, con rutas protegidas y middleware (`middleware.ts`) para asegurar el acceso.

Scripts y configuración: Archivos como `hash-password.ts` permiten operaciones específicas (por ejemplo, generar hashes bcrypt para contraseñas). Los archivos de configuración (`next.config.ts`, `tsconfig.json`, etc.) definen el entorno de desarrollo y despliegue.

Recursos estáticos: Imágenes y archivos públicos se almacenan en la carpeta `public/`.

VirtuStack

Gestión centralizada

Virtugem te permite controlar todas tus VMs desde un solo lugar, de manera sencilla y eficiente.

Lista de VMs

MySQL IA

En marcha

final

ID: 116

Tipo: QEMU

CPU: 2 cores

RAM: 2 MB

Disco: 0 GB

Uptime: 2d 2h

Consola

Reiniciar

Apagar

Eliminar

En marcha

MySQL

mysql-dev

ID: 106

Tipo: LXC

CPU: 1 cores

RAM: 1 MB

Disco: 1061 GB

Uptime: 8d 14h

AWX

1. Requisitos previos

- *Maquina debian 12* con acceso a internet

2. Instalación de paquetes

Antes de proceder al despliegue de AWX, es necesario preparar el entorno con herramientas esenciales, para ello instalamos los siguientes paquetes:

```
1 apt install -y curl wget vim git net-tools gnupg2 lsb-release ca-certificates apt-transport-https
```

3. Instalamos Kubernetes con k3s

En este caso optamos como orquestador por k3s, que es una distribución ligera y optimizada de Kubernetes.

```
1 curl -sfl https://get.k3s.io | sh -
```

Una vez instalado verificamos que funciona:

```
1 kubectl get nodes
```

4. Instalamos Kustomize

AWX Operator utiliza Kustomize para gestionar las configuraciones de despliegue. Para instalarlo:

```
1 curl -s "https://raw.githubusercontent.com/kubernetes-sigs/kustomize/master/hack/install_kustomize.sh" | bash
2 mv kustomize /usr/local/bin/
```

5. Instalamos AWX Operator

Para facilitar el despliegue y gestión de AWX dentro de Kubernetes, se emplea el AWX Operator. Este componente permite administrar la instancia de AWX de forma declarativa y sencilla.

Clonamos el repositorio oficial

```
1 git clone https://github.com/ansible/awx-operator.git
2 cd awx-operator
3 git checkout 2.14.0
```

Creamos el namespace de Kubernetes donde se alojará AWX

```
1 kubectl create namespace awx
```

Desplegamos el operador

```
1 make deploy NAMESPACE=awx
```



El comando make deploy usa Kustomize para instalar el operador.

6. Instalamos la instancia de AWX

Finalmente, se despliega la instancia de AWX dentro de Kubernetes mediante un fichero `awx.yml` que define los parámetros básicos de funcionamiento:

```
1  apiVersion: awx.ansible.com/v1beta1
2  kind: AWX
3  metadata:
4    name: awx
5    namespace: awx
6  spec:
7    service_type: nodeport
8    ingress_type: none
```

Se aplica la configuración al clúster:

```
1  kubectl apply -f awx.yml
```


7. Resultado final:

Bienvenido a

Inicie sesión

Usuario *

Contraseña *

Detalles

◀ Volver a Proyectos

Detalles

Acceso

Plantillas de

Último estado de la tarea

✔ Correctamente

Non

Organización

Default

Tip

URL de fuente de control ?

https://github.com/van
dreu82/ansible_rocks

Tie

Directorio de playbook ?

_14__oracle

Cre

Editar

Sincronizar

ELIMINAR

Ejemplo de proyecto conectado a github.

Detalles

◀ Volver a Plantillas

Detalles

Acceso

Notificación

Nombre

Oracle

Organización

Default

Entorno de ejecución ?

AWX EE (latest)

Nivel de detalle ?

0 (Normal)

Fraccionamiento de trabajos ?

1

Credenciales ?

SSH: Clave SSH

Variables ?

YAML

JSON

1

Editar

Ejecutar

ELIMINAR

router

✓ Correctamente

Stdout ▼



30 ok: [solo]

31



32 TASK [router : Instalar iptables-persistent] *****

33 ok: [solo]

34



35 TASK [router : Añadir regla MASQUERADE si no existe] *

36 changed: [solo]

37



38 TASK [router : Añadir FORWARD de LAN a WAN si no exist

39 changed: [solo]

40



41 TASK [router : Añadir FORWARD de WAN a LAN para conexi

42 changed: [solo]

43



44 TASK [router : Guardar reglas iptables persistentes] *

45 changed: [solo]

46



47 TASK [radvd : Instalar radvd] *****

48 ok: [solo]

49



50 TASK [radvd : Copiar configuración de radvd] *****

51 ok: [solo]

52



53 TASK [radvd : Habilitar y arrancar radvd] *****

54 ok: [solo]

55

56 PLAY RECAP *****

Salida

◀ Volver a Tareas

Detalles

Salida

testing



Ejecutándose

INICIAR

ping

Github

En este proyecto, GitHub se utiliza como repositorio remoto para almacenar los playbooks de Ansible, que son consumidos directamente por AWX para automatizar las tareas de despliegue y gestión de infraestructura.

Permite centralizar y versionar los playbooks de forma segura, garantizando que cualquier cambio realizado sea fácilmente auditable y recuperable.

Además, en GitHub hemos almacenado también la documentación generada a lo largo del desarrollo, ya que nos permite trabajar de forma sincronizada y facilita la colaboración, manteniendo un único punto de referencia actualizado y accesible.

Playbooks

Playbook de Oracle

Este playbook automatiza el despliegue de Oracle Database 23ai sobre Oracle Linux 9.5, instalando el software necesario, configurando la base de datos, el listener para las conexiones y una PDB instituto funcional con su respectivo esquema de base de datos.

Roles incluidos:

- **common:** Instala paquetes y configuraciones comunes para la máquina.
- **oracle_install:** Encargado de instalar y configurar Oracle Database, incluyendo esto la descarga, la instalación y la inicialización del servicio.
- **instituto_db:** Configura las variables de entorno, el listener para servir a la PDB instituto y crea la PDB instituto junto al user instituto_admin, que será el dueño del esquema de la base de datos que también se crea con un script.sql durante el rol.

Playbook de WordPress

Automatiza la instalación y configuración de dos servidores WordPress en Ubuntu Server, cada uno con su propia página de inicio personalizada y base de datos preparada para uso inmediato.

Roles incluidos:

- **common_db:** Configura la base de datos, crea usuarios y garantiza la conectividad segura entre WordPress y MySQL.
- **wordpress:** Despliega y configura WordPress, estableciendo la instalación con templates.
- **wp_db:** Gestiona la configuración de la base de datos para cada instancia de WordPress.

Playbook de HAProxy

Configura un servidor HAProxy para balancear la carga entre las dos instancias de WordPress, mejorando la disponibilidad y la capacidad de respuesta de la plataforma.

Roles incluidos:

- **haproxy:** Instala y configura el servicio HAProxy, creando las reglas de balanceo mediante un template con variables.

Playbook de Router (Kea y BIND)

Despliega un servidor router que integra DHCP (Kea) y DNS (BIND) para gestionar de forma centralizada las direcciones IP y la resolución de nombres, con soporte tanto para IPv4 como para IPv6.

Roles incluidos:

- **network-setup**: Aplica las configuraciones de red necesarias en la máquina, incluyendo la interfaz de red y el direccionamiento IP.
- **kea**: Configura el servidor DHCP con soporte para redes IPv4 e IPv6, permitiendo la asignación automática de direcciones IP.
- **bind**: Configura el servidor DNS con zonas directas e inversas, asegurando la resolución de nombres en la red interna.
- **router**: Gestiona la configuración de red, incluyendo el reenvío de paquetes y el enrutamiento necesario para interconectar las distintas subredes.
- **radvd**: Implementa el servicio de anuncios de router para IPv6, permitiendo tener una puerta de enlace IPv6.

Playbook de Migración de Máquinas Virtuales

Facilita la migración de máquinas virtuales desde un entorno local Debian/KVM a un entorno remoto Proxmox, garantizando la integridad y el funcionamiento tras el traslado.

Por supuesto. Aquí tienes una versión más técnica y precisa del mismo contenido:

Roles incluidos:

- **gather_vm_info**: Extrae información estructurada de las máquinas virtuales KVM a un fichero xml, incluyendo configuración de CPU, memoria, interfaces de red y dispositivos de almacenamiento. Esta información se almacena en formato xml para su posterior uso automatizado.
- **transfer_disks**: Detecta el primer vmid libre a partir de un umbral definido, crea el directorio correspondiente en el nodo Proxmox remoto, y transfiere los discos de las VMs, junto con los archivos de configuración xml usando rsync, asegurando la consistencia mediante sincronización diferencial.
- **proxmox_create_vm**: Automatiza la creación de máquinas virtuales en Proxmox a partir de los datos previamente recopilados. Define las VMs con los parámetros de hardware correctos, importa discos al almacenamiento LOCAL DE PROXMOX, y enlaza los discos a las máquinas.

Resultados

Se ha conseguido una integración completa y funcional entre el dashboard propio y AWX, permitiendo la gestión centralizada de máquinas virtuales y usuarios.

El sistema garantiza la seguridad de las credenciales mediante el uso de hash bcrypt y la transmisión segura de datos.

La automatización de tareas administrativas y la sincronización de datos han reducido el esfuerzo manual y el riesgo de errores humanos.

El dashboard ofrece una experiencia de usuario intuitiva y robusta, con validación de datos y mensajes de error claros.

Los objetivos generales y específicos han sido alcanzados, demostrando la viabilidad y utilidad de la solución propuesta.

Webgrafía:

- [Let's Encrypt](#)
- [DuckDNS](#)
- [Certbot](#)
- [Herramienta online para consultar registros TXT](#)
- [Proxmox](#)
- [AWX](#)

10. Anexos

- https://github.com/vandreu82/ansible_rocks
- https://github.com/Juaninux/Documentacion_TFG
- <https://gitlab.com/burruezo/pyapp>